On-chain Exploit Analysis

Version 1.0

Written by:

LAU LOK JING

August 6, 2026

Table of Contents

Attack Vector Analysis

Attack 1 - (ETH) 0x2c1a19982aa88bee8a5d9a5dfeb406f2bfe1cfc1213f20e91d91ce3b55c86cc5

Transaction hash: 0x2c1a19982aa88bee8a5d9a5dfeb406f2bfe1cfc1213f20e91d91ce3b55c86cc5

Root Cause:

Lack of Input Validation and Improper Slippage Handling in `TokenRewards::depositFromPairedLpToken` from Peapods Finance.

The `depositFromPairedLpToken` accepts a user-supplied `_slippageOverride` parameter, which is used directly to compute the `amountOutMinimum` for the Uniswap V3 `swapRouter::exactInputSingle` which allows `amountOutMinimum = 0`.

In TokenRewards.sol:

```
    function depositFromPairedLpToken(
        uint256 _amountTknDepositing,
@>      uint256 _slippageOverride
    ) public override {
        ...
@>  uint256 _slippage = _slippageOverride > 0
      ? _slippageOverride
      : _rewardsSwapSlippage;
    try
      ISwapRouter(V3_ROUTER).exactInputSingle(
        ISwapRouter.ExactInputSingleParams({
          tokenIn: PAIRED_LP_TOKEN,
          tokenOut: rewardsToken,
          fee: REWARDS_POOL_FEE,
          recipient: address(this),
          deadline: block.timestamp,
          amountIn: _amountTkn,
@>        amountOutMinimum: (_amountOut * (1000 - _slippage)) / 1000,
          sqrtPriceLimitX96: 0
        })
      )
```

When `_slippageOverride` is set to `999`, the multiplier becomes `(1000 - 999)= 1`, so `amountOutMinimum = _amountOut / 1000` resulting in 0.1% of the expected output, effectively disabling slippage protection because any price. Additionally, the `depositFromPairedLpToken` is **`public`** and did not validate the `_slippageOverride` value, allowing an attacker to set it arbitrarily.

By combining this with a price manipulation attack front-run, the attacker can force the contract to swap its `PAIRED_LP_TOKEN` balance at an extremely unfavorable rate, extracting value.

Buggy Contract:

- Name: Peapods Finance
- Address: 0x7d48D6D775FaDA207291B37E3eaA68Cc865bf9Eb
- Entry point: `TokenRewards::depositFromPairedLpToken`

Victim:

1. The `TokenRewards` contract itself, Peapods Finance sold `356.891 pOHM -> 192.905 PEAS` at a manipulated price, resulting in a loss of value for the protocol.
2. Stakers and the protocol treasury, since the rewards pool is decreased.

Attack Flow:

1. Flash Swap (Uniswap V2)
    - The attacker contract calls `UniswapV2Pair.swap` to borrow **9,420 pOHM** from the `pOHM/PEAS V2 pool`.
    - This triggers the `uniswapV2Call` callback on the attacker contract.
2. Price Manipulation (Uniswap V3 Front-Run)
    - Inside the callback, the attacker swaps the borrowed **9,420 pOHM** for **17,022.705 PEAS** on Uniswap V3 pool.
    - This large swap drives the `pOHM/PEAS` price **downward**, creating a manipulated price state.
3. Exploit Call to `TokenRewards`
    - The attacker calls `TokenRewards.depositFromPairedLpToken(_amountTknDepositing = 0, _slippageOverride = 999)`.
    - `depositFromPairedLpToken` uses the contract's existing balance of `PAIRED_LP_TOKEN` (**~= 356.891 pOHM**) as amountIn for Uniswap V3.
    - `amountOutMinimum` is configured to `0.1%` of the expected output, effectively removing all slippage protection.
    - The contract executes a UniswapV3 `swapRouter::exactInputSingle` to sell **356.89 pOHM** for **192.90 PEAS** at the manipulated price.
4. Arbitrage Exit (Uniswap V3 Back-Run)
    - After the `depositFromPairedLpToken` call ends, the attacker swaps the **17,022.705 PEAS** they received in step 2 back for **9,589.45 pOHM** UniswapV3 `swapRouter::exactInputSingle`.
5. Repay Flash Swap
    - At the end of the `uniswapV2Call` callback, the attacker transfers **9,448 pOHM** (fees included) back to the V2 pool to repay the flash loan.
    - The remaining **9,589.45 - 9,448 = 141.113 pOHM** is the attacker's profit.

Attack 2 - (BSC) 0x8501a9d34fc28bee21fd08505b3e4b1b0a4aec3a5496b508a290531d8eb25281

Transaction hash: 0x8501a9d34fc28bee21fd08505b3e4b1b0a4aec3a5496b508a290531d8eb25281

Root Cause:

Missing Slippage Protection in Multiple Swap functions combined with the Fee-On-Transfer Mechanics of the `ADAcash` Token.

Specifically:

1. The victim contract `0x2d70d62` has a `public` `depositBNB` function that swaps `WBNB` for `ADAcash` without any slippage protection by calling `PancakeRouter::swapExactETHForTokensSupportingFeeOnTransferTokens` with `amountOutMin = 0`.
2. `ADAcash` is a fee-on-transfer token that charges a 15% fee on every transfer, which accumulates in the token contract.
3. The `ADAcash` contract automatically swaps accumulated fees for `ADA` and `WBNB` without slippage protection:
    - Most collected fees are swapped for `ADA`
    - ~13% are converted to `WBNB` to boost `ADAcash`/`WBNB` pool liquidity
4. All `internal` swap functions (`swapAndSendToFee`, `swapAndLiquify`, `swapAndSendDividends`) call `PancakeRouter::swapExactETHForTokensSupportingFeeOnTransferTokens` with `amountOutMin = 0` resulting in zero slippage protection entirely.

This creates a*double-sandwich vulnerability where the attacker can sandwich the victim's unprotected swap and sandwich the `ADAcash` contract's own fee-swap mechanism to extract value twice.

Buggy Contract:

Contract	Entry point
`0x2d70d62`	`depositBNB` function swaps WBNB -> ADAcash with `amountOutMin = 0` (no slippage protection)
`ADAcash`	Internal fee-swap functions call PancakeSwap router with `amountOutMin = 0`

Victim:

1. The unverified contract `0x2d70d62`, which lost approximately 225.995 BNB.
2. `ADAcash` token holders and liquidity providers, as the attack manipulated the token's price and drained value from the protocol's fee-swap mechanism.

Attack Flow:

S1: Flash Loan Acquisition

1. Attacker contract (`0xc011232F9891bF698C7dfF8bff95D79ED9Eb4e74`) calls Pancake V3 Pool 1 `flash` to borrow 6,431.521 `WBNB`.
2. Attacker calls Pancake V3 Pool 2 flash loan to borrow an additional 5,568.478 `WBNB`.

S2: Price Manipulation (First Sandwich)

1. Attacker swaps `WBNB` -> `ADAcash`, driving the ADAcash price up. The 15% fee on this transfer accumulates in the `ADAcash` contract.
2. Attacker swaps `WBNB` -> `ADA`, driving the ADA price up.
3. Attacker triggers the victim contract's `depositBNB` function, which swaps `WBNB` -> `ADAcash` without slippage protection. This is the first sandwiched swap, further spiking ADAcash's price.

S3: Profit Extraction (Second Sandwich)

1. Attacker sells `ADAcash` to profit from the first sandwich. However, selling `ADAcash` also triggers the 15% fee.
2. To minimize the impact of the 15% fee, the attacker splits the sale into multiple smaller swaps. Each sale triggers the `ADAcash` contract to swap accumulated fees (now substantial due to the price manipulation).
3. These fee-swaps constitute the second sandwiched swap since they also lack slippage protection, the attacker profits from them as well. These sales further drive up the `ADAcash` price.
4. Attacker sells `ADAcash` to lock in profits from the second sandwich.

S4: Repayment and Profit

1. Attacker repays flash loan 1: 5,671.262 `WBNB`.
2. Attacker repays flash loan 2: 6,432.164 `WBNB`.
3. Attacker retains ~186.30 `WBNB` as profits.

Source Code References:

1. ADAcash
2. PancakeV3Pool
3. PancakeRouter
4. Victim 0x2d70d62
5. UniswapV2Router02

Attack 3 - (ETH) 0x04975648e0db631b0620759ca934861830472678dae82b4bed493f1e1e3ed03a

Transaction: 0x04975648e0db631b0620759ca934861830472678dae82b4bed493f1e1e3ed03a

Root Cause:

Calldata Corruption in the Old Version of the 1inch Settlement contract - Fusion V1 protocol.

The vulnerability exists in the `_settleOrder` function, which processes order suffixes. When constructing the calldata for the `fillOrderTo` call, the contract uses an unsafe pointer calculation that can be manipulated by an attacker to cause an integer underflow, effectively allowing the attacker to overwrite critical fields in the calldata structure.

The vulnerable code calculates the suffix offset as:

```
1     function _settleOrder(bytes calldata data, address resolver,
          uint256 totalFee, bytes memory tokensAndAmounts) private {
2         ...
3         assembly {
4             function memcpy(dst, src, len) {
5                 pop(staticcall(gas(), 0x4, src, len, dst, len))
6             }
7
8             let interactionLengthOffset := calldataload(add(data.offset
                  , 0x40))
9             let interactionOffset := add(interactionLengthOffset, 0x20)
10 @>          let interactionLength := calldataload(add(data.offset,
       interactionLengthOffset))
11
12            { // stack too deep
13                let target := shr(96, calldataload(add(data.offset,
                      interactionOffset)))
14                if or(lt(interactionLength, 20), iszero(eq(target,
                      address()))) {
15                    mstore(0, _WRONG_INTERACTION_TARGET_SELECTOR)
16                    revert(0, 4)
17                }
18            }
19
20            // Copy calldata and patch interaction.length
21            let ptr := mload(0x40)
22            mstore(ptr, _FILL_ORDER_TO_SELECTOR)
23            calldatacopy(add(ptr, 4), data.offset, data.length)
24 @>          mstore(add(add(ptr, interactionLengthOffset), 4), add(
       interactionLength, suffixLength))
25
26            {  // stack too deep
27                // Append suffix fields
```

```
28 @>               let offset := add(add(ptr, interactionOffset),
       interactionLength)
29                  mstore(add(offset, 0x04), totalFee)
30                  mstore(add(offset, 0x24), resolver)
31                  mstore(add(offset, 0x44), calldataload(add(order, 0x40)
                        ))  // takerAsset
32                  mstore(add(offset, 0x64), rateBump)
33                  mstore(add(offset, 0x84), takingFeeData)
34                  let tokensAndAmountsLength := mload(tokensAndAmounts)
35                  memcpy(add(offset, 0xa4), add(tokensAndAmounts, 0x20),
                        tokensAndAmountsLength)
36                  mstore(add(offset, add(0xa4, tokensAndAmountsLength)),
                        tokensAndAmountsLength)
37              }
38
39              ...
40          }
41      }
```

Because `interactionLength` is a full 32-byte word controlled by the attacker, setting it to a large value (e.g., `0xffff...fe00 = -512`) causes the pointer to underflow, writing the suffix to a different location in memory.

This allows the attacker to:

1. Create a normal order swapping a tiny amount for millions of dollars
2. Pad it with null bytes
3. Specify an invalid `interactionLength` value to corrupt the calldata
4. Inject a fake suffix that gets parsed as the actual order data

The resulting calldata corruption allows the attacker to call arbitrary resolver contracts and execute unauthorized swaps.

Buggy Contract:

Primary vulnerable contract: Old version of the 1inch Settlement contract (Fusion V1 protocol) `0xA88800CD213dA5Ae406ce248380802BD53b47647`, specifically the `_settleOrder` function in the Yul implementation.

Key vulnerable library: `Settlement.sol`.

Victim:

`0xb02f39e382c90160eb816de5e0e428ac771d77b5`, a market maker integrated with 1inch Fusion, .

Attack Flow:

1. The attacker created a normal-looking order that swapped a tiny amount (a few wei) for millions of dollars worth of tokens.
2. The attacker exploited the unsafe pointer calculation in `_settleOrder` by:
    1. Padding the order with null bytes,
    2. Setting `interactionLength` to an invalid value (`0xffff...fe00 = −512`), causing the pointer to underflow,
    3. Adding a fake suffix structure as an interaction. This caused the real suffix to be written to the padding allocated before the fake suffix structure. The interaction structure was then treated as a suffix by the `DynamicSuffix` library.
3. The corrupted calldata caused the contract to call an arbitrary and unauthorised resolver with the attacker's fabricated order data.
4. The unauthorized `resolveOrders` call executed the attacker's fabricated order, draining funds from the resolver contract.